

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde  
xer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_56dfabe6-f8c\agent-  
a589313cb66ea7816.jsonl`  
- SHA-256: `e1f8d0f9725436c9922c9503481332036234b838da2d88d142392bbb92f26927`  
- Source modified: `2026-07-07T00:18:44+00:00`  
- Imported at: `2026-07-08T16:00:05+00:00`  
- project: `wf_56dfabe6-f8c`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-07T00:17:09.272Z)

Reviewing GitHub PR #57 "F7: add /why explainability command" (branch feat/f7-why-command -> main). ADR 0012 F7 (final phase). +295/-2, 5 files.

Checked-out copy at C:/Users/avidu/Projects/_wt-pr57. Run repros:

PYTHONPATH=C:/Users/avidu/Projects/_wt-pr57/src

C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>.

WHAT F7 DOES:

```
- command_service.py: a /why command (regex allows "/why" alone or "/why <arg>").  
_handle_why(conn, user_id, arg, cfg): if arg is None or  
    not arg.strip().isdecimal() or int(arg) < 1 -> Usage reply. Else item =  
db.get_content_item_for_account(conn, user_id, item_id);  
    if item is None -> "Content item `<id>` was not found for your account."; else decisions =  
db.list_policy_decisions_for_content(item.id);  
    if decisions -> _format_policy_decisions(item, decisions); else ->  
_format_missing_policy_decision(item, current_cfg).  
- db.py get_content_item_for_account: SELECT * FROM content_items WHERE id=? AND account_id=?  
(the account-scoped IDOR guard).  
- _format_policy_decisions renders each decision + stage (reason + metadata via  
_format_json_value = json.dumps(default=str, ensure_ascii=False), each value truncated to 500  
chars).  
- _format_missing_policy_decision: replays the CURRENT FilterConfig via FilterEngine over a  
VideoMeta built from persisted content_item fields  
    (duration/width/height/size/mime only), with a caveat that historical config changes may  
differ.
```

VERIFIED by the main reviewer: gate green (560 passed @ 93.99%); IDOR handled ? account A doing /why on account B item returns "not found for your account"

with NO leak of B channel/caption/url/domain; B sees own trace; malformed (/why, /why abc, /why -1, /why 0) -> Usage. Tests cover cross-account access + scoping.

Ground rules: READ code, cite file:line. Run repros. Report ONLY defects in THIS PR diff. No praise. Concrete failure scenario, ranked.

Severity: high (access-control bypass / cross-account leak, or crash on user input), medium (real edge bug), low (rendering/limit/note).

DIMENSION: access control / IDOR / cross-account leakage ? try HARD to find a leak. Is EVERY data-returning path gated by the account check? list_policy_decisions_for_content(item.id) is NOT account-scoped itself ? safe only because the item was ownership-checked first? Any path where another account decisions/trace/source/caption/url become visible? Does the fallback ever run for a non-owned item? Does the "not found" message reveal existence of another account item (not-exists vs not-owned oracle)? Is user_id truly the requester account in the wired adapter (commands.py passes user.id, gated by message.out + peer_id==user.id)? Can user_id be attacker-controlled? Run cross-account repros. A real leak = high.

2. assistant (2026-07-07T00:17:11.854Z)

I'll review PR #57 for access control / IDOR / cross-account leakage. Let me start by reading the relevant files.

3. user (2026-07-07T00:17:14.568Z)

```
12b4442 feat(commands): add why explainability command
6149c38 feat(policy): persist policy decision traces (#55)
da0ab47 feat(policy): add deep inspection budget boundary (#54)
a7cbbda feat(policy): add URL reputation stage (#53)
5cd65d1 feat(policy): add caption safety stage (#52)
---DIFF STAT---
src/tg_video_filter/command_service.py | 120 ++++++
src/tg_video_filter/commands.py        |   2 +
src/tg_video_filter/db.py              |  16 +++
tests/test_command_service.py          | 146 ++++++
tests/test_db.py                      |  13 +++
5 files changed, 295 insertions(+), 2 deletions(-)
```

4. user (2026-07-07T00:17:15.274Z)

```
src/tg_video_filter/commands.py
src/tg_video_filter/command_service.py
src/tg_video_filter/db.py
```

5. assistant (2026-07-07T00:17:17.209Z)

Let me read the key files in this PR.

6. user (2026-07-07T00:17:18.227Z)

```
1      """Provider-independent command parsing and execution (ADR 0011, Track D).
2
3      Extracts command logic from the Telethon adapter (`commands.py`) so the
4      same ``/set``, ``/show`` commands work identically through a Bot API
5      front-door. The service is pure: strings in, ``CommandResult`` out ? no
6      Telethon objects, no network calls.
7
8      See ``docs/adr/0007-inline-config-commands.md`` for the command model and
9      ``docs/adr/0008-bot-api-onboarding-front-door.md`` for the Bot API motivation.
10     """
11
12     from __future__ import annotations
13
14     import json
15     import logging
16     import re
17     from dataclasses import dataclass
18
19     import aiosqlite
20
21     from tg_video_filter import db
22     from tg_video_filter.filters import FilterEngine, VideoMeta
23     from tg_video_filter.models import ContentItem, FilterConfig, PolicyDecision, Source
24
25     log = logging.getLogger(__name__)
26
27     # Key into the shared engine_holder dict (imported by commands.py's adapter).
28     ENGINE_KEY = "engine"
29
30     # --- regex patterns -----
31
32     _SIZE_RE = re.compile(r"^/set\s+size\s+(\d+)\s*([kmgT]?b?)$", re.IGNORECASE)
```

```

33     _LENGTH_RE = re.compile(r"^/set\s+(?:max\s+)?length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
34     _MIN_LENGTH_RE = re.compile(r"^/set\s+min\s+length\s+(\d+)\s*([smh]?)$", re.IGNORECASE)
35     _MIN_RES_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+(\d+)\s*p?$",
re.IGNORECASE)
36     _SHOW_CONFIG_RE = re.compile(r"^/show\s+config$", re.IGNORECASE)
37     _SHOW_COMMANDS_RE = re.compile(r"^/show\s+commands$", re.IGNORECASE)
38     _WHY_RE = re.compile(r"^/why(?:\s+(.))*?$", re.IGNORECASE)
39     _EXCLUDE_RE = re.compile(r"^/set\s+exclude\s+(gif|round|nosound)\s+(on|off)$",
re.IGNORECASE)
40     _EXCLUDE_HELP_RE = re.compile(r"^/set\s+exclude\s+", re.IGNORECASE)
41     _MIN_RES_HELP_RE = re.compile(r"^/set\s+min\s+(?:resolution|res)\s+", re.IGNORECASE)
42
43
44     # --- source scanning regex (ADR 0009, Track E) -----
45
46     _WATCH_RE = re.compile(r"^/watch\s+(.)*$", re.IGNORECASE)
47     _UNWATCH_RE = re.compile(r"^/unwatch\s+(.)*$", re.IGNORECASE)
48     _CHANNELS_RE = re.compile(r"^/channels$", re.IGNORECASE)
49     _WATCHLIST_RE = re.compile(r"^/watchlist$", re.IGNORECASE)
50
51     # --- unit parsing -----
52
53
54     def parse_size_mb(value_str: str, unit: str) -> int:
55         """Parse a size value + unit string into mebibytes."""
56         value = int(value_str)
57         unit = unit.lower().rstrip("b")
58         multipliers: dict[str, int] = {"": 1, "m": 1, "k": 1, "g": 1024, "t": 1024 * 1024}
59         if unit in multipliers:
60             raw = value * multipliers[unit]
61             if unit == "k":
62                 raw = max(1, raw // 1024)
63             return raw
64         raise ValueError(f"Unknown size unit: {unit}")
65
66
67     def parse_duration_s(value_str: str, unit: str) -> int:
68         """Parse a duration value + unit string into seconds."""
69         value = int(value_str)
70         unit = unit.lower()
71         multipliers: dict[str, int] = {"": 1, "s": 1, "m": 60, "h": 3600}
72         if unit in multipliers:
73             return value * multipliers[unit]
74         raise ValueError(f"Unknown time unit: {unit}")
75
76
77     # --- formatting helpers -----
78
79
80     def _format_duration(seconds: int | None) -> str:
81         if seconds is None:
82             return "unlimited"
83         if seconds == 0:
84             return "0s"
85         if seconds < 60:
86             return f"{seconds}s"
87         if seconds < 3600:
88             m, s = divmod(seconds, 60)
89             return f"{m}m{s}s" if s else f"{m}m"
90         h, rem = divmod(seconds, 3600)
91         m, s = divmod(rem, 60)
92         parts = f"{h}h"
93         if m:
94             parts += f"{m}m"
95         if s:

```

```

96         parts += f"{s}s"
97     return parts
98
99
100 def _format_size(mb: int | None) -> str:
101     if mb is None:
102         return "unlimited"
103     if mb == 0:
104         return "0MB"
105     if mb >= 1024:
106         gb = mb / 1024
107         if gb == int(gb):
108             return f"{int(gb)}GB"
109         return f"{gb:.1f}GB"
110     return f"{mb}MB"
111
112
113 def format_config(cfg: FilterConfig) -> str:
114     """Format a ``FilterConfig`` as a readable message (Telegram markdown)."""
115     lines = [
116         "? **Current Filter Configuration**",
117         "",
118         f"?? **Duration:** {_format_duration(cfg.min_duration_s)} - "
119         f"{_format_duration(cfg.max_duration_s)}",
120         f"? **Min resolution:** {'?' + str(cfg.target_height) + 'p' if "
121         f"cfg.target_height else 'disabled'}",
122         f"? **Max file size:** {_format_size(cfg.max_size_mb)}",
123         f"? **Delivery mode:** `{cfg.delivery_mode.value}`",
124         f"? **Delivery target:** `{cfg.delivery_target}`",
125         "",
126         "? **Content exclusions:**",
127         f"    GIF/animated: {'?' excluded' if cfg.exclude_gif else '? allowed'}",
128         f"    Round videos: {'?' excluded' if cfg.exclude_round else '? allowed'}",
129         f"    Silent (no audio): {'?' excluded' if cfg.exclude_nosound else '? allowed'}",
130     ]
131     return "\n".join(lines)
132
133 def format_commands() -> str:
134     """Return a help message listing all available commands."""
135     return "\n".join(
136         [
137             "? **Available Commands**",
138             "",
139             "? **Filter config**",
140             "`/show config` ? Show current filter settings",
141             "`/set size <N><unit>` ? Set max file size (e.g. `/set size 10MB`)",
142             "    Units: `MB` (default), `GB`, `KB` ? case-insensitive, optional trailing "
143             "`B`",
144             "`/set length <N><unit>` ? Set max video duration (e.g. `/set length 15m`)",
145             "    or `/set max length <N><unit>` ? same as above",
146             "    Units: `s` (default), `m`, `h` ? case-insensitive",
147             "`/set min length <N><unit>` ? Set min video duration (e.g. `/set min length "
148             "5s`)",
149             "    Units: `s` (default), `m`, `h` ? case-insensitive",
150             "`/set min resolution <N>p` ? Set min video height (e.g. `/set min "
151             "resolution 720p`)",
152             "    or `/set min res <N>p` ? same as above",
153             "",
154             "? **Content exclusions** (all off by default)",
155             "`/set exclude gif on|off` ? Exclude Telegram GIFs/animated documents",
156             "`/set exclude round on|off` ? Exclude circular video messages",
157             "`/set exclude nosound on|off` ? Exclude videos without audio\n"
158             "    Note: GIFs are usually silent; `exclude nosound` will catch most GIFs "
159             "too.",
160         ]
161     )

```

```

155         "",
156         "? **Backfill**",
157         "`/backfill [N][hdw]` ? Scan last N hours/days/weeks of channel history
(default 90d)",
158         "`/stop` ? Cancel running backfill",
159         "",
160         "? **Source scanning**",
161         "`/channels` ? List all known sources with watch status",
162         "`/watch <id|@name>` ? Start watching a source for new videos",
163         "`/unwatch <id|@name>` ? Stop watching a source",
164         "`/watchlist` ? Show only watched sources",
165         "",
166         "? **Explainability**",
167         "`/why <content_item_id>` ? Explain why an item passed or failed",
168         "",
169         "\u2139\u2013 **Help**",
170         "`/show commands` ? Show this help message",
171     ]
172 )
173
174
175 def _format_json_value(value: object, max_len: int = 500) -> str:
176     text = json.dumps(value, default=str, ensure_ascii=False, sort_keys=True)
177     if len(text) <= max_len:
178         return text
179     return f"{text[: max_len - 1]}?"
180
181
182 def _content_item_to_video_meta(item: ContentItem) -> VideoMeta:
183     """Build a metadata replay input from fields persisted on content_items."""
184     return VideoMeta(
185         duration_s=item.duration_s,
186         width=item.width,
187         height=item.height,
188         size_bytes=item.size_bytes,
189         mime_type=item.mime_type,
190     )
191
192
193 def _format_policy_decisions(item: ContentItem, decisions: list[PolicyDecision]) -> str:
194     status = "PASSED" if decisions[-1].passed else "REJECTED"
195     lines = [
196         f"? **Why content item `{item.id}`?",
197         "",
198         f"Final policy result: **{status}**",
199         f"Provider message: `{item.provider}:{item.provider_message_id}`",
200     ]
201     if item.link_url:
202         lines.append(f"Link: {item.link_url}")
203     if item.text:
204         lines.append(f"Text: {_format_json_value(item.text, max_len=160)}")
205
206     for decision in decisions:
207         decision_status = "PASSED" if decision.passed else "REJECTED"
208         lines.extend(
209             [
210                 "",
211                 f"Policy `{decision.policy_id}`: **{decision_status}**",
212                 f"Evaluated: `{decision.evaluated_at.isoformat()}`",
213             ]
214         )
215     if not decision.trace:
216         lines.append("- No stage trace recorded.")
217     continue
218     for stage in decision.trace:

```

```

219         stage_name = str(stage.get("stage_name", "unknown"))
220         stage_passed = bool(stage.get("passed"))
221         reason = str(stage.get("reason", "no reason recorded"))
222         icon = "?" if stage_passed else "?"
223         lines.append(f"- {icon} `{stage_name}` ? {reason}")
224         metadata = stage.get("metadata")
225         if isinstance(metadata, dict) and metadata:
226             lines.append(f"    metadata: `{_format_json_value(metadata)}`")
227     return "\n".join(lines)
228
229
230 def _format_missing_policy_decision(item: ContentItem, current_cfg: FilterConfig) ->
str:
231     recorded_status = "accepted" if item.matched else "rejected"
232     replay = FilterEngine(current_cfg).evaluate(_content_item_to_video_meta(item))
233     replay_status = "accepted" if replay.matched else "rejected"
234     lines = [
235         f"? **Why content item `{item.id}`?***",
236         "",
237         "No policy-stage decision trace was recorded for this item.",
238         f"Recorded item status: **{recorded_status}**.",
239         "",
240         "Current metadata-filter replay:",
241         f"- Result with current config: **{replay_status}**",
242     ]
243     if replay.reasons:
244         lines.extend(f"- {reason}" for reason in replay.reasons)
245     else:
246         lines.append("- No metadata filters are currently configured.")
247     lines.extend(
248         [
249             "",
250             "This fallback uses the current filter config and persisted metadata only; "
251             "historical config changes can make it differ from the original decision.",
252         ]
253     )
254     return "\n".join(lines)
255
256
257 # --- command result -----
258
259
260 @dataclass
261 class CommandResult:
262     """The output of executing a command ? provider-independent.
263
264     The adapter (Telethon or Bot API) uses ``reply_text`` to send the response
265     and ``config_changed`` to decide whether to hot-reload the filter engine.
266     """
267
268     reply_text: str = ""
269     config_changed: bool = False
270
271
272 # --- command service -----
273
274
275 class CommandService:
276     """Provider-independent command execution.
277
278     Parses a command string against an account's current config and DB,
279     returning a ``CommandResult``. No Telethon objects involved ? pure string
280     in, effects out. The Telethon adapter (``commands.py``) handles the
281     transport (checking message.out, sending the reply); the Bot API adapter
282     (future) calls ``handle()`` with the ``from.id`` from the update.

```

```

283
284 Usage::
285
286     svc = CommandService()
287     result = await svc.handle(user_id, text, conn, current_cfg)
288     if result.config_changed:
289         engine_holder["engine"] = FilterEngine(current_cfg)
290     await send_reply(result.reply_text)
291 """"
292
293 # -- public API -----
294
295 async def handle(
296     self,
297     user_id: int,
298     text: str,
299     conn: aiosqlite.Connection,
300     current_cfg: FilterConfig,
301 ) -> CommandResult:
302     """Parse and execute a command text, returning the reply + change flag."""
303     text = text.rstrip()
304
305     # --- /set size <value><unit> -----
306     m = _SIZE_RE.match(text)
307     if m:
308         value_str, unit = m.group(1), m.group(2) or ""
309         # NOTE: ValueError is unreachable here ? the regex already validates
310         # the unit against [kmgT]?b?, so parse_size_mb can never raise.
311         # Kept for defensive parity with the original commands.py refactor.
312         try:
313             new_mb = parse_size_mb(value_str, unit)
314         except ValueError:
315             return CommandResult("? Invalid size format. Use e.g. `/set size
10MB`.")
316         if new_mb < 1:
317             return CommandResult("? Size must be at least 1 MB (`FilterConfig`
constraint).")
318         cfg = await self._update(conn, user_id, current_cfg, max_size_mb=new_mb)
319         return CommandResult(
320             f"? Max file size set to
**[_format_size(new_mb)]**.\n\n{format_config(cfg)}",
321             config_changed=True,
322         )
323
324     # --- /set length <value><unit> -----
325     m = _LENGTH_RE.match(text)
326     if m:
327         value_str, unit = m.group(1), m.group(2) or ""
328         try:
329             new_secs = parse_duration_s(value_str, unit)
330         except ValueError:
331             return CommandResult("? Invalid duration format. Use e.g. `/set length
15m`.")
332         min_secs = current_cfg.min_duration_s or 0
333         if new_secs < min_secs:
334             return CommandResult(
335                 f"? Max duration ({_format_duration(new_secs)}) must be "
336                 f">= min duration ({_format_duration(min_secs)}).")
337         )
338         cfg = await self._update(conn, user_id, current_cfg,
max_duration_s=new_secs)
339         return CommandResult(
340             f"? Max duration set to
**[_format_duration(new_secs)]**.\n\n{format_config(cfg)}",
341             config_changed=True,

```

```

342         )
343
344     # --- /set min length <value><unit> -----
345     m = _MIN_LENGTH_RE.match(text)
346     if m:
347         value_str, unit = m.group(1), m.group(2) or ""
348         try:
349             new_min = parse_duration_s(value_str, unit)
350         except ValueError:
351             return CommandResult("? Invalid duration format. Use e.g. `/set min
length 5s`.")
352         max_secs = current_cfg.max_duration_s or 0
353         if max_secs > 0 and new_min > max_secs:
354             return CommandResult(
355                 f"? Min duration ({_format_duration(new_min)}) must be "
356                 f"<= max duration ({_format_duration(max_secs)})."
357             )
358         cfg = await self._update(conn, user_id, current_cfg, min_duration_s=new_min)
359         return CommandResult(
360             f"? Min duration set to
**{_format_duration(new_min)}**.\n\n{format_config(cfg)}",
361             config_changed=True,
362         )
363
364     # --- /set min resolution <N>p -----
365     m = _MIN_RES_RE.match(text)
366     if m:
367         new_height = int(m.group(1))
368         if new_height < 1:
369             return CommandResult("? Resolution must be at least 1p.")
370         cfg = await self._update(conn, user_id, current_cfg,
target_height=new_height)
371         return CommandResult(
372             f"? Min resolution set to **{new_height}p**.\n\n{format_config(cfg)}",
373             config_changed=True,
374         )
375
376     if _MIN_RES_HELP_RE.match(text):
377         return CommandResult(
378             "? Usage: `/set min resolution <N>p` (e.g. `/set min resolution 720p`)"
379         )
380
381     # --- /set exclude <gif|round|nosound> <on|off> -----
382     m = _EXCLUDE_RE.match(text)
383     if m:
384         what = m.group(1).lower()
385         state = m.group(2).lower()
386         enabled = state == "on"
387         labels = {"gif": "GIF/animated", "round": "round videos", "nosound": "silent
videos"}
388         field_map = {
389             "gif": "exclude_gif",
390             "round": "exclude_round",
391             "nosound": "exclude_nosound",
392         }
393         cfg = await self._update(conn, user_id, current_cfg, **{field_map[what]:
enabled})
394         return CommandResult(
395             f"? {labels[what]} exclusion: **{'ON' if enabled else 'OFF'}**."
396             f"\n\n{format_config(cfg)}",
397             config_changed=True,
398         )
399
400     if _EXCLUDE_HELP_RE.match(text):
401         return CommandResult(

```



```

402         "? Usage: `/set exclude <gif|round|nosound> <on|off>`\n"
403         "Example: `/set exclude gif on`"
404     )
405
406     # --- /show config -----
407     if _SHOW_CONFIG_RE.match(text):
408         return CommandResult(format_config(current_cfg))
409
410     # --- source scanning commands (ADR 0009, Track E) -----
411
412     m = _WATCH_RE.match(text)
413     if m:
414         return await self._handle_watch(conn, user_id, m.group(1).strip(),
current_cfg)
415
416     m = _UNWATCH_RE.match(text)
417     if m:
418         return await self._handle_unwatch(conn, user_id, m.group(1).strip(),
current_cfg)
419
420     if _CHANNELS_RE.match(text):
421         return await self._handle_channels(conn, user_id)
422
423     if _WATCHLIST_RE.match(text):
424         return await self._handle_watchlist(conn, user_id)
425
426     # --- /why <content_item_id> -----
427     m = _WHY_RE.match(text)
428     if m:
429         return await self._handle_why(conn, user_id, m.group(1), current_cfg)
430
431     # --- /show commands -----
432     if _SHOW_COMMANDS_RE.match(text):
433         return CommandResult(format_commands())
434
435     # Not a recognised command ? return empty result (noise).
436     return CommandResult()
437
438     # -- source scanning handlers (ADR 0009, Track E) -----
439
440     async def _handle_watch(
441         self, conn:aiosqlite.Connection, user_id: int, target: str, _cfg: FilterConfig
442     ) -> CommandResult:
443         """Parse `/watch <@username|channel_id>` and toggle `watched=1`."""
444         src = await self._resolve_source(conn, user_id, target)
445         if src is None:
446             return CommandResult(
447                 f"? Source `{target}` not found. Use `/channels` to list "
448                 f"available sources.\n\nSources are discovered as messages "
449                 f"arrive; a channel you haven't received traffic from may not "
450                 f"appear yet."
451             )
452         await db.set_source_watched(conn, user_id, src.id, True) # type: ignore[arg-
type]
453         label = src.title or src.provider_source_id
454         return CommandResult(f"? Now watching **{label}**
({src.provider_source_id}).")
455
456     async def _handle_unwatch(
457         self, conn:aiosqlite.Connection, user_id: int, target: str, _cfg: FilterConfig
458     ) -> CommandResult:
459         """Parse `/unwatch <@username|channel_id>` and toggle `watched=0`."""
460         src = await self._resolve_source(conn, user_id, target)
461         if src is None:
462             return CommandResult(

```

```

463         f"? Source `{target}` not found. Use `/channels` to list sources."
464     )
465     await db.set_source_watched(conn, user_id, src.id, False) # type: ignore[arg-
type]
466     label = src.title or src.provider_source_id
467     return CommandResult(f"? Stopped watching **{label}**
({src.provider_source_id}).")
468
469     async def _handle_channels(self, conn: aiomysqlite.Connection, user_id: int) ->
CommandResult:
470         """List all known sources with watch status."""
471         sources = await db.list_account_sources(conn, user_id)
472         if not sources:
473             return CommandResult(
474                 "? **No sources yet.**\n\n"
475                 "Sources are discovered when a new message arrives from a "
476                 "channel. Run `/backfill` to discover historical channels, "
477                 "or wait for new traffic."
478             )
479         lines = ["? **All sources**", ""]
480         for s in sources:
481             icon = "?" if s.watched else "?"
482             label = s.title or s.provider_source_id
483             lines.append(f"{icon} `{s.provider_source_id}` ? {label}")
484         return CommandResult("\n".join(lines))
485
486     async def _handle_watchlist(self, conn: aiomysqlite.Connection, user_id: int) ->
CommandResult:
487         """List only watched sources."""
488         sources = await db.list_account_sources(conn, user_id, watched_only=True)
489         if not sources:
490             return CommandResult(
491                 "? **No watched sources.**\n\n"
492                 "Use `/channels` to see available sources, then `/watch <id>` "
493                 "to start scanning."
494             )
495         lines = ["? **Watched sources**", ""]
496         for s in sources:
497             label = s.title or s.provider_source_id
498             lines.append(f"? `{s.provider_source_id}` ? {label}")
499         return CommandResult("\n".join(lines))
500
501     async def _handle_why(
502         self,
503         conn: aiomysqlite.Connection,
504         user_id: int,
505         item_id_text: str | None,
506         current_cfg: FilterConfig,
507     ) -> CommandResult:
508         """Explain a recorded content item's policy decision."""
509         if item_id_text is None or not item_id_text.strip().isdecimal():
510             return CommandResult("? Usage: `/why <content_item_id>`)
511         item_id = int(item_id_text.strip())
512         if item_id < 1:
513             return CommandResult("? Usage: `/why <content_item_id>`)
514
515         item = await db.get_content_item_for_account(conn, user_id, item_id)
516         if item is None:
517             return CommandResult(f"? Content item `{item_id}` was not found for your
account.")
518
519         assert item.id is not None
520         decisions = await db.list_policy_decisions_for_content(conn, item.id)
521         if decisions:
522             return CommandResult(_format_policy_decisions(item, decisions))

```

```

523         return CommandResult(_format_missing_policy_decision(item, current_cfg))
524
525     async def _resolve_source(
526         self, conn: aiosqlite.Connection, user_id: int, target: str
527     ) -> Source | None:
528         """Resolve a user-provided source identifier to a ``Source`` row.
529
530         Accepts numeric channel id, ``@username``, or channel title.
531         ``@`` prefix is stripped before lookup.
532         """
533         target = target.removeprefix("@")
534         # Try by provider-native numeric id (the common case).
535         src = await db.get_source_by_provider_id(conn, user_id, "telegram", target)
536         if src is not None:
537             return src
538         # Try by @username.
539         src = await db.get_source_by_provider_id(conn, user_id, "telegram",
540 f"@{target}")
541         if src is not None:
542             return src
543         # Try by title (case-insensitive substring match).
544         all_srcs = await db.list_account_sources(conn, user_id)
545         target_lower = target.lower()
546         for s in all_srcs:
547             if s.title and target_lower in s.title.lower():
548                 return s
549         return None
550
551     # -- internal -----
552
553     async def _update(
554         self,
555         conn: aiosqlite.Connection,
556         user_id: int,
557         current: FilterConfig,
558         max_size_mb: int | None = None,
559         max_duration_s: int | None = None,
560         min_duration_s: int | None = None,
561         target_height: int | None = None,
562         exclude_gif: bool | None = None,
563         exclude_round: bool | None = None,
564         exclude_nosound: bool | None = None,
565     ) -> FilterConfig:
566         """Apply partial updates to the account's config, persist, and return."""
567         updates: dict[str, object] = {}
568         if max_size_mb is not None:
569             updates["max_size_mb"] = max_size_mb
570         if max_duration_s is not None:
571             updates["max_duration_s"] = max_duration_s
572         if min_duration_s is not None:
573             updates["min_duration_s"] = min_duration_s
574         if target_height is not None:
575             updates["target_height"] = target_height
576         if exclude_gif is not None:
577             updates["exclude_gif"] = exclude_gif
578         if exclude_round is not None:
579             updates["exclude_round"] = exclude_round
580         if exclude_nosound is not None:
581             updates["exclude_nosound"] = exclude_nosound
582         cfg = current.model_copy(update=updates) if updates else current
583         await db.save_filter_config(conn, user_id, cfg)
584         log.info(
585             "config_updated user=%s max_size_mb=%s min_duration_s=%s max_duration_s=%s "
586             "target_height=%s exclude_gif=%s exclude_round=%s exclude_nosound=%s",
587             user_id,

```

```

587         cfg.max_size_mb,
588         cfg.min_duration_s,
589         cfg.max_duration_s,
590         cfg.target_height,
591         cfg.exclude_gif,
592         cfg.exclude_round,
593         cfg.exclude_nosound,
594     )
595     return cfg
596
597
598     __all__ = [
599         "ENGINE_KEY",
600         "CommandResult",
601         "CommandService",
602         "_WHY_RE",
603         "_format_duration",
604         "_format_size",
605         "format_config",
606         "format_commands",
607         "parse_duration_s",
608         "parse_size_mb",
609     ]
610

```

7. user (2026-07-07T00:17:20.190Z)

```

diff --git a/src/tg_video_filter/commands.py b/src/tg_video_filter/commands.py
index 3aea1f6..06db189 100644
--- a/src/tg_video_filter/commands.py
+++ b/src/tg_video_filter/commands.py
@@ -41,6 +41,7 @@ from tg_video_filter.command_service import ( # noqa: E402, F401
     _UNWATCH_RE,
     _WATCH_RE,
     _WATCHLIST_RE,
+    _WHY_RE,
     _format_duration,
     _format_size,
     format_commands,
@@ -142,6 +143,7 @@ __all__ = [
     "_EXCLUDE_HELP_RE",
     "_MIN_RES_HELP_RE",
     "_SIZE_RE",
+    "_WHY_RE",
     "_format_duration",
     "_format_size",
     "_update_config",
diff --git a/src/tg_video_filter/db.py b/src/tg_video_filter/db.py
index 6cdd65f..0d1069d 100644
--- a/src/tg_video_filter/db.py
+++ b/src/tg_video_filter/db.py
@@ -1285,6 +1285,21 @@ async def get_content_item_by_dedup_key(
     return _row_to_content_item(row) if row else None

+async def get_content_item_for_account(
+    conn: aiosqlite.Connection,
+    account_id: int,
+    content_item_id: int,
+) -> ContentItem | None:
+    """Read a content item by id, scoped to an account for user-facing lookups."""
+    conn.row_factory = aiosqlite.Row
+    async with conn.execute(
+        "SELECT * FROM content_items WHERE id = ? AND account_id = ?",
+        (content_item_id, account_id),

```

```

+     ) as cur:
+         row = await cur.fetchone()
+         return _row_to_content_item(row) if row else None
+
+
+ # --- Policy repos (ADR 0011, Option 2A) -----

```

```

@@ -1973,6 +1988,7 @@ __all__ = [
    "insert_content_item",
    "get_content_item_by_message",
    "get_content_item_by_dedup_key",
+   "get_content_item_for_account",
    "upsert_policy",
    "get_default_policy",
    "load_policy_config",

```

8. assistant (2026-07-07T00:17:23.435Z)

The core ``/why`` path is well-gated: ``get_content_item_for_account`` scopes by ``account_id``, and ``list_policy_decisions_for_content`` runs only after ownership is confirmed. Let me examine the wiring in ``commands.py`` (how ``user_id`` gets there), ``list_policy_decisions_for_content``, and the `account_id/content_items` relationship to check for any leak vector.

9. user (2026-07-07T00:17:25.004Z)

```

1     """Config commands ? Telethon adapter over the provider-independent `CommandService`.
2
3     This module is a thin Telethon adapter. It handles Telethon-specific transport
4     (checking `message.out`, `peer_id`, sending replies) and delegates command
5     parsing and config mutation to `command_service.CommandService` (ADR 0011, Track D).
6
7     Re-exports regex patterns and unit-parsers for backwards-compatibility with
8     existing tests (`tests/test_commands.py`).
9
10    See `docs/adr/0007-inline-config-commands.md` for the command model and
11    `docs/adr/0008-bot-api-onboarding-front-door.md` for the Bot API front-door.
12    """
13
14    from __future__ import annotations
15
16    import logging
17    from collections.abc import Callable
18
19    import aiosqlite
20    from telethon import TelegramClient
21    from telethon.errors import RPCError
22
23    from tg_video_filter import db
24    from tg_video_filter.command_service import ENGINE_KEY, CommandResult, CommandService
25    from tg_video_filter.filters import FilterEngine
26    from tg_video_filter.models import FilterConfig, User
27
28    log = logging.getLogger(__name__)
29
30    from tg_video_filter.command_service import ( # noqa: E402, F401
31        _CHANNELS_RE,
32        _EXCLUDE_HELP_RE,
33        _EXCLUDE_RE,
34        _LENGTH_RE,
35        _MIN_LENGTH_RE,
36        _MIN_RES_HELP_RE,
37        _MIN_RES_RE,
38        _SHOW_COMMANDS_RE,
39        _SHOW_CONFIG_RE,

```

```

40     _SIZE_RE,
41     _UNWATCH_RE,
42     _WATCH_RE,
43     _WATCHLIST_RE,
44     _WHY_RE,
45     _format_duration,
46     _format_size,
47     format_commands,
48     format_config,
49     parse_duration_s,
50     parse_size_mb,
51 )
52
53
54 def make_config_command_handler(
55     user: User,
56     conn: aiosqlite.Connection,
57     engine_holder: dict[str, FilterEngine],
58 ) -> Callable:
59     svc = CommandService()
60
61     async def on_command(event: object) -> None:
62         message = getattr(event, "message", None)
63         if message is None:
64             return
65         if not getattr(message, "out", False):
66             return
67         peer = getattr(message, "peer_id", None)
68         peer_user_id = getattr(peer, "user_id", None) if peer else None
69         if peer_user_id != user.id:
70             return
71
72         text: str = getattr(message, "message", "") or ""
73         client: TelegramClient = getattr(event, "client", None) # type:
ignore[assignment]
74
75         # Defer DB load ? skip non-command self-messages (review finding #2).
76         if not text.startswith("/"):
77             return
78
79         current_cfg = await db.load_filter_config(conn, user.id)
80         result: CommandResult = await svc.handle(user.id, text, conn, current_cfg)
81
82         if result.config_changed:
83             new_cfg = await db.load_filter_config(conn, user.id)
84             engine_holder[ENGINE_KEY] = FilterEngine(new_cfg)
85
86         if result.reply_text:
87             await _reply(client, message, result.reply_text)
88
89     return on_command
90
91
92 # --- backwards-compat helpers (test_commands.py references) -----
93
94
95 async def _update_config(
96     conn: aiosqlite.Connection,
97     user_id: int,
98     max_size_mb: int | None = None,
99     max_duration_s: int | None = None,
100     min_duration_s: int | None = None,
101     target_height: int | None = None,
102     exclude_gif: bool | None = None,
103     exclude_round: bool | None = None,

```

```

104         exclude_nosound: bool | None = None,
105     ) -> FilterConfig:
106         """Deprecated wrapper ? delegates to ``CommandService._update``.
107
108         Kept for backwards-compatibility with ``tests/test_commands.py`` which
109         imports ``_update_config`` from here. New code should call
110         :meth:`CommandService.handle` which returns a :class:`CommandResult`.
111         """
112         svc = CommandService()
113         cfg = await db.load_filter_config(conn, user_id)
114         return await svc._update(
115             conn,
116             user_id,
117             cfg,
118             max_size_mb=max_size_mb,
119             max_duration_s=max_duration_s,
120             min_duration_s=min_duration_s,
121             target_height=target_height,
122             exclude_gif=exclude_gif,
123             exclude_round=exclude_round,
124             exclude_nosound=exclude_nosound,
125         )
126
127
128     async def _reply(client: TelegramClient, to_message: object, text: str) -> None:
129         try:
130             await client.send_message("me", text)
131         except (OSError, TimeoutError, RPCError):
132             log.warning("config_reply_failed", exc_info=True)
133
134
135     __all__ = [
136         "ENGINE_KEY",
137         "_EXCLUDE_RE",
138         "_LENGTH_RE",
139         "_MIN_LENGTH_RE",
140         "_MIN_RES_RE",
141         "_SHOW_CONFIG_RE",
142         "_SHOW_COMMANDS_RE",
143         "_EXCLUDE_HELP_RE",
144         "_MIN_RES_HELP_RE",
145         "_SIZE_RE",
146         "_WHY_RE",
147         "_format_duration",
148         "_format_size",
149         "_update_config",
150         "format_commands",
151         "format_config",
152         "make_config_command_handler",
153         "parse_duration_s",
154         "parse_size_mb",
155     ]
156

```

10. user (2026-07-07T00:17:26.705Z)

```

52:     account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
99:     account_id    INTEGER NOT NULL REFERENCES accounts(id),
107:     UNIQUE (account_id, provider, provider_source_id)
112:     account_id    INTEGER NOT NULL REFERENCES accounts(id),
129:     UNIQUE (account_id, source_id, provider_message_id),
130:     UNIQUE (account_id, dedup_key)
135:     account_id    INTEGER NOT NULL REFERENCES accounts(id),
141:     UNIQUE (account_id, name)
146: ON policies(account_id) WHERE is_default = 1;

```

```

150:     account_id INTEGER NOT NULL REFERENCES accounts(id),
157:     UNIQUE (account_id, provider, target)
162:     account_id INTEGER NOT NULL REFERENCES accounts(id),
169:     UNIQUE (account_id, source_id, destination_id)
419:     account_id INTEGER PRIMARY KEY REFERENCES accounts(id),
434:     INSERT INTO telegram_sessions (account_id, phone, session_path, enabled, created_at)
543:         (account_id, provider, provider_source_id, title, kind, watched,
558:         (account_id, provider, provider_source_id, kind, watched, seen_first_at)
575:         (account_id, source_id, provider, provider_message_id, content_type,
591:     ON s.account_id = v.user_id
606:     GROUP BY account_id, document_id
627:     GROUP BY account_id, size_bytes, duration_s, height, width
634: #     this row. UNIQUE(account_id, name) dedups idempotent re-runs.
637:     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
651:     "ON policies(account_id) WHERE is_default = 1"
707:     INSERT OR IGNORE INTO destinations (account_id, provider, target, kind, is_output,
created_at)
733:         (account_id, source_id, policy_id, destination_id, enabled, priority,
created_at)
734:     SELECT s.account_id, s.id, p.id, d.id, 1, 0,
737:     JOIN policies p ON p.account_id = s.account_id AND p.is_default = 1
738:     JOIN destinations d ON d.account_id = s.account_id
746:     WHERE fcd.user_id = s.account_id
762:     ON ci.account_id = vd.user_id
768:     JOIN destinations d ON d.account_id = vd.user_id
770:     ON rr.account_id = vd.user_id
843:     account_id=row["account_id"],
879:     account_id=row["account_id"],
893:     account_id=row["account_id"],
916:     account_id=row["account_id"],
942:     account_id=row["account_id"],
955:     account_id=row["account_id"],
1004:async def get_account(conn: aiosqlite.Connection, account_id: int) -> Account | None:

```

11. assistant (2026-07-07T00:17:29.354Z)

Now let me look at `list\_policy\_decisions\_for\_content` and how policy_decisions link to content_items / account, plus the schema for policy_decisions.

12. user (2026-07-07T00:17:32.223Z)

```

186:CREATE TABLE IF NOT EXISTS policy_decisions (
779:async def _migration_005_policy_decisions(conn: aiosqlite.Connection) -> None:
783:     CREATE TABLE IF NOT EXISTS policy_decisions (
890:def _row_to_content_item(row: aiosqlite.Row) -> ContentItem:
925:def _row_to_policy_decision(row: aiosqlite.Row) -> PolicyDecision:
928:         raise ValueError("policy_decisions.trace_json must decode to a JSON array")
1424:         """INSERT INTO policy_decisions
1453:         """SELECT * FROM policy_decisions
1461:async def list_policy_decisions_for_content(
1468:         """SELECT * FROM policy_decisions
1998:     "list_policy_decisions_for_content",

```

13. user (2026-07-07T00:17:35.097Z)

```

186     CREATE TABLE IF NOT EXISTS policy_decisions (
187         id INTEGER PRIMARY KEY AUTOINCREMENT,
188         content_item_id INTEGER NOT NULL REFERENCES content_items(id),
189         policy_id INTEGER NOT NULL REFERENCES policies(id),
190         passed INTEGER NOT NULL,
191         trace_json TEXT NOT NULL,
192         evaluated_at TEXT NOT NULL,
193         UNIQUE (content_item_id, policy_id)
194     );

```



```

195     """
196
197     # Note: the ``idx_videos_user_document`` partial unique index is created by
198     # migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
199     # which is present on fresh tables but only added to legacy v1 DBs by the
200     # migration; putting it in ``_SCHEMA`` would break ``executescript`` on legacy
201     # DBs that don't yet have the column.
202
203
204     async def init_db(db_path: str) -> None:
205         """Open the database at ``db_path`` and create/migrate the schema.
206
207         Safe to call on every startup. Applies the base schema (``IF NOT EXISTS``)
208         then runs any pending migrations via the ``user_version`` ladder.
209         """
210         async with aiosqlite.connect(db_path) as conn:
211             await _apply_schema(conn)
212
213
214     async def _apply_schema(conn: aiosqlite.Connection) -> None:
215         # Enable WAL mode (concurrent reads+writes), foreign key enforcement,

```

14. user (2026-07-07T00:17:35.737Z)

```

1453         """SELECT * FROM policy_decisions
1454             WHERE content_item_id = ? AND policy_id = ?""",
1455         (content_item_id, policy_id),
1456     ) as cur:
1457         row = await cur.fetchone()
1458         return _row_to_policy_decision(row) if row else None
1459
1460
1461     async def list_policy_decisions_for_content(
1462         conn: aiosqlite.Connection,
1463         content_item_id: int,
1464     ) -> list[PolicyDecision]:
1465         """Return all policy decisions recorded for a content item."""
1466         conn.row_factory = aiosqlite.Row
1467         async with conn.execute(
1468             """SELECT * FROM policy_decisions
1469                 WHERE content_item_id = ?
1470                 ORDER BY evaluated_at, id""",
1471             (content_item_id,),
1472         ) as cur:
1473             rows = await cur.fetchall()
1474             return [_row_to_policy_decision(row) for row in rows]
1475
1476
1477     # --- Destination / RoutingRule / DeliveryAttempt repos (ADR 0011, Track C) --
1478
1479
1480     async def upsert_destination(conn: aiosqlite.Connection, dest: Destination) ->
Destination:
1481         """Insert or update a destination row by (account, provider, target)."""
1482         await conn.execute(

```

15. assistant (2026-07-07T00:17:41.052Z)

Now I understand the data model. `list\_policy\_decisions\_for\_content(item.id)` filters only by `content\_item\_id`, but since `item` was already ownership-checked via `get\_content\_item\_for\_account`, and `content\_item\_id` is a globally unique PK (autoincrement), the decisions returned all belong to the owned item. This is safe ? the ownership check gates the ID.

Let me verify the account scoping is airtight by running a cross-account repro. First, let me

check the test file to understand DB setup helpers.

16. user (2026-07-07T00:17:42.461Z)

```
diff --git a/tests/test_command_service.py b/tests/test_command_service.py
index 320f8a5..11a2be8 100644
--- a/tests/test_command_service.py
+++ b/tests/test_command_service.py
@@ -17,7 +17,7 @@ from tg_video_filter.command_service import (
    parse_duration_s,
    parse_size_mb,
)
-from tg_video_filter.models import FilterConfig, Source
+from tg_video_filter.models import ContentItem, FilterConfig, Policy, Source

USER_ID = 111

@@ -237,6 +237,7 @@ def test_format_commands_includes_all_sections() -> None:
    assert "/channels" in text
    assert "/watch " in text
    assert "/watchlist" in text
+    assert "/why" in text

# --- should_ingest_source (telethon_client gate, extracted for testability) ---
@@ -376,3 +377,146 @@ async def test_list_account_sources_watched_only(
    watched = await db.list_account_sources(conn, USER_ID, watched_only=True)
    assert len(watched) == 1
    assert watched[0].provider_source_id == "-10"
+
+
+## --- /why explainability (ADR 0012, Track F7) -----
+
+
+async def _seed_content_item(
+    conn: aiosqlite.Connection,
+    *,
+    account_id: int = USER_ID,
+    provider_message_id: str = "42",
+    matched: bool = False,
+    height: int | None = 720,
+) -> ContentItem:
+    src = await _seed_source(conn, account_id, f"-100{provider_message_id}", "Trace Source")
+    assert src.id is not None
+    item = await db.insert_content_item(
+        conn,
+        ContentItem(
+            account_id=account_id,
+            source_id=src.id,
+            provider="telegram",
+            provider_message_id=provider_message_id,
+            duration_s=120,
+            height=height,
+            size_bytes=10 * 1024 * 1024,
+            dedup_key=f"tg:msg:{account_id}:{provider_message_id}",
+            matched=matched,
+        ),
+    )
+    assert item.id is not None
+    return item
+
+
+async def _seed_policy(conn: aiosqlite.Connection, account_id: int = USER_ID) -> Policy:
+    policy = await db.upsert_policy(
+        conn,
```

```

+         Policy(
+             account_id=account_id,
+             name="default",
+             config_json={},
+             is_default=True,
+         ),
+     )
+     assert policy.id is not None
+     return policy
+
+
+async def test_why_renders_policy_decision_trace(
+    conn: aiomysqlite.Connection, svc: CommandService
+) -> None:
+    item = await _seed_content_item(conn, matched=False)
+    policy = await _seed_policy(conn)
+    assert item.id is not None
+    assert policy.id is not None
+    await db.record_policy_decision(
+        conn,
+        content_item_id=item.id,
+        policy_id=policy.id,
+        passed=False,
+        trace=[
+            {
+                "stage_name": "topic-preferences",
+                "passed": True,
+                "reason": "topic preferences disabled",
+                "metadata": {},
+            },
+            {
+                "stage_name": "url-reputation",
+                "passed": False,
+                "reason": "blocked domain matched: bad.example",
+                "metadata": {"matched_blocklist_domain": "bad.example"},
+            },
+        ],
+    )
+
+    result = await svc.handle(USER_ID, f"/why {item.id}", conn, FilterConfig())
+
+    assert "Final policy result: **REJECTED**" in result.reply_text
+    assert "`url-reputation`" in result.reply_text
+    assert "blocked domain matched: bad.example" in result.reply_text
+    assert "matched_blocklist_domain" in result.reply_text
+    assert result.config_changed is False
+
+
+async def test_why_missing_policy_decision_uses_metadata_fallback(
+    conn: aiomysqlite.Connection, svc: CommandService
+) -> None:
+    item = await _seed_content_item(conn, matched=False, height=720)
+    assert item.id is not None
+
+    result = await svc.handle(
+        USER_ID,
+        f"/why {item.id}",
+        conn,
+        FilterConfig(target_height=1080),
+    )
+
+    assert "No policy-stage decision trace was recorded" in result.reply_text
+    assert "Recorded item status: **rejected**" in result.reply_text
+    assert "height 720 < target 1080" in result.reply_text
+    assert "current filter config" in result.reply_text

```

```

+
+
+@pytest.mark.parametrize("text", ["/why", "/why abc", "/why -1"])
+async def test_why_malformed_input_shows_usage(
+    conn: aiosqlite.Connection, svc: CommandService, text: str
+) -> None:
+    result = await svc.handle(USER_ID, text, conn, FilterConfig())
+
+    assert "Usage: `/why <content_item_id>`" in result.reply_text
+    assert result.config_changed is False
+
+
+
+async def test_why_missing_item_shows_clear_response(
+    conn: aiosqlite.Connection, svc: CommandService
+) -> None:
+    result = await svc.handle(USER_ID, "/why 9999", conn, FilterConfig())
+
+    assert "not found for your account" in result.reply_text
+    assert result.config_changed is False
+
+
+
+async def test_why_rejects_cross_account_access(
+    conn: aiosqlite.Connection, svc: CommandService
+) -> None:
+    other_user = 222
+    await db.upsert_user(
+        conn,
+        db.User(id=other_user, phone="+15555550200"), # type: ignore[attr-defined]
+    )
+    await db.save_filter_config(conn, other_user, FilterConfig())
+    item = await _seed_content_item(
+        conn,
+        account_id=other_user,
+        provider_message_id="77",
+        matched=True,
+    )
+    assert item.id is not None
+
+    result = await svc.handle(USER_ID, f"/why {item.id}", conn, FilterConfig())
+
+    assert "not found for your account" in result.reply_text
+    assert "Trace Source" not in result.reply_text

```

17. assistant (2026-07-07T00:17:47.066Z)

Now let me write a cross-account repro to verify the IDOR guard holds, including the case where account A has a decision trace on B's item ID and the "not-exists vs not-owned oracle" question.

18. user (2026-07-07T00:17:49.271Z)

```

8:from __future__ import annotations
10:import aiosqlite
11:import pytest
13:from tg_video_filter import db
14:from tg_video_filter.command_service import (
20:from tg_video_filter.models import ContentItem, FilterConfig, Policy, Source
25:@pytest.fixture
26:async def conn() -> aiosqlite.Connection:
27:    """An in-memory SQLite connection with a seeded user and default config."""
28:    c = await aiosqlite.connect(":memory:")
39:@pytest.fixture
40:def svc() -> CommandService:

```

19. user (2026-07-07T00:17:53.128Z)

```

@pytest.fixture
async def conn() -> aiosqlite.Connection:
    """An in-memory SQLite connection with a seeded user and default config."""
    c = await aiosqlite.connect(":memory:")
    await db._apply_schema(c)
    await db.upsert_user(
        c,
        db.User(id=USER_ID, phone="+15555550100"), # type: ignore[attr-defined]
    )
    await db.save_filter_config(c, USER_ID, FilterConfig())
    yield c
    await c.close()

@pytest.fixture
def svc() -> CommandService:
    return CommandService()

```

--- unit parser tests -----

```

@pytest.mark.parametrize(
    "value, unit, expected",
    [
        ("10", "MB", 10),
        ("1", "GB", 1024),
        ("500", "KB", 1),
        ("5", "", 5),
        ("2", "TB", 2 * 1024 * 1024),
    ],
)
def test_parse_size_mb(value: str, unit: str, expected: int) -> None:
    assert parse_size_mb(value, unit) == expected

def test_parse_size_mb_unknown_unit_raises() -> None:
    with pytest.raises(ValueError):
        parse_size_mb("10", "XB")

@pytest.mark.parametrize(
    "value, unit, expected",
    [
        ("15", "m", 900),
        ("2", "h", 7200),
        ("90", "s", 90),
        ("5", "", 5),
    ],
)
def test_parse_duration_s(value: str, unit: str, expected: int) -> None:
    assert parse_duration_s(value, unit) == expected

def test_parse_duration_s_unknown_unit_raises() -> None:
    with pytest.raises(ValueError):
        parse_duration_s("10", "x")

```

--- /set size -----

```

async def test_set_size(conn: aiosqlite.Connection, svc: CommandService) -> None:

```

```

result = await svc.handle(USER_ID, "/set size 100MB", conn, FilterConfig())
assert "100MB" in result.reply_text
assert result.config_changed is True

```

20. assistant (2026-07-07T00:17:55.493Z)

Now let me write a comprehensive cross-account IDOR repro covering the key attack scenarios.

21. assistant (2026-07-07T00:18:12.000Z)

```

import asyncio
import aiosqlite
from tg_video_filter import db
from tg_video_filter.command_service import CommandService
from tg_video_filter.models import ContentItem, FilterConfig, Policy, Source

```

```

ATTACKER = 111
VICTIM = 222

```

```

async def seed_source(conn, account_id, psid, title):
    return await db.upsert_source(
        conn,
        Source(account_id=account_id, provider="telegram",
              provider_source_id=psid, title=title, kind="channel", watched=False),
    )

```

```

async def seed_item(conn, account_id, pmid, matched, link=None, text=None):
    src = await seed_source(conn, account_id, f"-100{pmid}", f"SecretChan-{account_id}")
    item = await db.insert_content_item(
        conn,
        ContentItem(account_id=account_id, source_id=src.id, provider="telegram",
              provider_message_id=pmid, duration_s=120, height=720,
              size_bytes=10*1024*1024, dedup_key=f"tg:{account_id}:{pmid}",
              matched=matched, link_url=link, text=text),
    )
    return item

```

```

async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    await db.upsert_user(conn, db.User(id=ATTACKER, phone="+1111"))
    await db.upsert_user(conn, db.User(id=VICTIM, phone="+2222"))
    await db.save_filter_config(conn, ATTACKER, FilterConfig())
    await db.save_filter_config(conn, VICTIM, FilterConfig())
    svc = CommandService()

    # Victim item with sensitive link/text + a policy decision trace
    v_item = await seed_item(conn, VICTIM, "77", matched=True,
        link="https://secret.victim.example/leak",
        text="VICTIM_SECRET_CAPTION")
    v_pol = await db.upsert_policy(conn, Policy(account_id=VICTIM, name="default",
        config_json={}, is_default=True))
    await db.record_policy_decision(conn, content_item_id=v_item.id, policy_id=v_pol.id,
        passed=True, trace=[{"stage_name": "url-reputation",
"passed": True, "reason": "VICTIM_TRACE_REASON", "metadata": {"domain": "secret.victim.example"}}])

    print("=== Scenario 1: attacker /why on victim item id ===")
    r = await svc.handle(ATTACKER, f"/why {v_item.id}", conn, FilterConfig())
    print(repr(r.reply_text))
    for leak in ["secret.victim.example", "VICTIM_SECRET_CAPTION", "VICTIM_TRACE_REASON",
"SecretChan-222", "/leak"]:
        assert leak not in r.reply_text, f"LEAK: {leak}"

```

```

print(" -> no victim data leaked. OK")

print("\n=== Scenario 2: not-exists vs not-owned oracle ===")
r_owned = await svc.handle(ATTACKER, f"/why {v_item.id}", conn, FilterConfig()) # exists,
not owned
r_ghost = await svc.handle(ATTACKER, "/why 999999", conn, FilterConfig()) # does not
exist
print("owned-by-other:", repr(r_owned.reply_text))
print("nonexistent :", repr(r_ghost.reply_text))
print(" identical:", r_owned.reply_text == r_ghost.reply_text)

print("\n=== Scenario 3: attacker owns an item, gets own trace (control) ===")
a_item = await seed_item(conn, ATTACKER, "88", matched=False)
r = await svc.handle(ATTACKER, f"/why {a_item.id}", conn, FilterConfig())
print("attacker sees own item:", "Why content item" in r.reply_text)

print("\n=== Scenario 4: shared content_item_id space -- decisions keyed only by
content_item_id ===")
print("victim item id:", v_item.id, "attacker item id:", a_item.id, "(distinct PKs, no
collision)")

await conn.close()

asyncio.run(main())

```

22. user (2026-07-07T00:18:12.028Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/7dc3da51-af38-47b6-83f6-fdda83b60664/scratchpad/repro_idor.py (file state is current in your context ? no need to Read it back)

23. user (2026-07-07T00:18:16.935Z)

```

=== Scenario 1: attacker /why on victim item id ===
'? Content item `1` was not found for your account.'
-> no victim data leaked. OK

=== Scenario 2: not-exists vs not-owned oracle ===
owned-by-other: '? Content item `1` was not found for your account.'
nonexistent   : '? Content item `999999` was not found for your account.'
identical: False

=== Scenario 3: attacker owns an item, gets own trace (control) ===
attacker sees own item: True

=== Scenario 4: shared content_item_id space -- decisions keyed only by content_item_id ===
victim item id: 1 attacker item id: 2 (distinct PKs, no collision)

```

24. assistant (2026-07-07T00:18:23.220Z)

The core IDOR guard holds. Scenario 2 shows the messages differ only because the ID is echoed back (both say "not found for your account" ? same template, same semantics). The item-exists-but-not-owned response is textually identical to the not-exists response except for the echoed ID the attacker already typed. There is no not-exists vs not-owned oracle: both return "not found for your account", so the attacker cannot distinguish "item exists but belongs to someone else" from "item does not exist." No leak.

Now let me examine the wiring question more carefully: can `user\_id` be attacker-controlled? The handler uses `user.id` (the account owner, bound at handler construction), gated by `message.out` and `peer\_id.user\_id == user.id`. Let me verify `user` is not derived from the incoming message.

25. user (2026-07-07T00:18:24.651Z)

```
src/tg_video_filter/commands.py:54:def make_config_command_handler(
src/tg_video_filter/commands.py:152:     "make_config_command_handler",
src/tg_video_filter/manager.py:123:     from tg_video_filter.commands import ENGINE_KEY,
make_config_command_handler
src/tg_video_filter/manager.py:170:         config_handler = make_config_command_handler(
Binary file src/tg_video_filter/__pycache__/commands.cpython-313.pyc matches
Binary file src/tg_video_filter/__pycache__/manager.cpython-313.pyc matches
```

26. user (2026-07-07T00:18:27.058Z)

```
160         client=client,
161         user=user,
162         conn=conn,
163         engine_holder=engine_holder,
164         delivery_mode=cfg.delivery_mode,
165         delivery_target=cfg.delivery_target,
166     )
167     client.add_event_handler(backfill_handler, events.NewMessage(chats=["me"]))
168
169     # Register the /set and /show config command handler on Saved Messages.
170     config_handler = make_config_command_handler(
171         user=user,
172         conn=conn,
173         engine_holder=engine_holder,
174     )
175     client.add_event_handler(config_handler, events.NewMessage(chats=["me"]))
176
177     log.info("client_connected account=%s", session.account_id)
178     try:
179         await client.run_until_disconnected()
180     finally:
181         log.info("client_disconnected account=%s", session.account_id)
182         await client.disconnect()
183
184
```

27. assistant (2026-07-07T00:18:34.989Z)

`user` is bound per-account at client startup (one Telethon client per account session), not derived from message content. `user.id` passed to `svc.handle` is the authenticated account owner. The `message.out` + `peer\_id.user\_id == user.id` checks ensure only the account's own Saved Messages self-commands are processed. `user\_id` is not attacker-controlled.

The access-control dimension is clean. The item is ownership-checked before `list\_policy\_decisions\_for\_content(item.id)`, `content\_item\_id` is a globally-unique PK so the returned decisions all belong to the owned item, the fallback path also runs only after ownership check, and there's no not-exists/not-owned oracle.

Let me do one final check: confirm there is no other data-returning path in `\_handle\_why` and that `item.id` (used unassert-guarded) matches the requested id ? a mismatch could theoretically fetch a different item's decisions. `get\_content\_item\_for\_account` does `WHERE id=? AND account\_id=?`, so `item.id == item\_id` always. No issue.

Now let me quickly confirm the tests pass for this dimension and there are no import errors.

28. user (2026-07-07T00:18:38.795Z)

src\tg_video_filter\delivery.py	102	102	48	0	0%	13-283
src\tg_video_filter\filters__init__.py	39	2	18	7	84%	43, 63->65,
65->67, 67->69, 69->71, 71->73, 74						
src\tg_video_filter\filters\base.py	26	0	0	0	100%	
src\tg_video_filter\filters\content_type.py	23	11	12	2	40%	20-22, 31-33,
40-44						
src\tg_video_filter\filters\duration.py	15	3	6	3	71%	22, 25, 27

src\tg_video_filter\filters\resolution.py	13	3	6	3	68%	22, 24, 27
src\tg_video_filter\filters\size.py	18	5	6	3	67%	22, 24, 27-28, 34
src\tg_video_filter\manager.py	36	36	8	0	0%	26-204
src\tg_video_filter\models.py	171	10	8	1	91%	379-383, 393, 433, 441-443
src\tg_video_filter\policy.py	415	415	152	0	0%	8-850
src\tg_video_filter\telethon_client.py	84	84	34	0	0%	16-514

TOTAL 1757 1128 506 50 31%

FAIL Required test coverage of 80.0% not reached. Total coverage: 30.80%

7 passed, 41 deselected in 0.82s

29. assistant (2026-07-07T00:18:44.602Z)

All 7 `/why` tests pass (the coverage failure is expected when running a subset). The IDOR/access-control dimension is solid.

Let me do a last targeted check on the fallback path for a subtle leak:

`\_format\_missing\_policy\_decision` calls `FilterEngine(current\_cfg).evaluate(...)` where `current\_cfg` is the **requester's** config. Since the item was already ownership-checked, replaying the requester's own config over their own item's metadata is correct ? no cross-account config bleed. And the "Recorded item status" comes from `item.matched` which belongs to the owned item. Clean.

I found no access-control / IDOR / cross-account leakage defects in this PR diff. Reporting empty findings for this dimension.

30. user (2026-07-07T00:18:44.606Z)

Structured output provided successfully